

SPEC-1-Online Food Ordering App

Background

This document outlines the step-by-step design of an **online food ordering application**, starting from first principles and progressively deepening into system architecture, data models, APIs, scalability, and deployment.

The goal is to build a **production-ready MVP** that enables customers to browse restaurants, view menus, place orders, and track delivery, while allowing restaurants and delivery partners to manage operations efficiently.

We will evolve this document iteratively as understanding increases. Each section will be validated before moving deeper, ensuring clarity and avoiding overload.

Initial Assumptions (can be adjusted): - Mobile-first product (Android/iOS), with a web admin panel - Multi-restaurant marketplace model (like Swiggy/Uber Eats) - MVP scope before advanced optimizations (AI, heavy analytics) - Cloud-native architecture

Requirements

The requirements are defined using **MoSCoW prioritization** to clearly separate what is essential for the MVP versus what can be added later.

Must Have (MVP)

- User registration and authentication (Customer, Restaurant Owner, Delivery Partner, Admin)
- Browse restaurants by location
- View restaurant menus and item details
- Add items to cart
- Place food orders
- Online payment (single payment gateway)
- Order status tracking (Placed → Preparing → Picked Up → Delivered)
- Restaurant order management dashboard
- Delivery partner order assignment and status updates
- Admin panel to manage users, restaurants, and orders

Should Have

- Search restaurants and food items
- Ratings and reviews for restaurants
- Order history for users
- Basic notifications (order status updates)
- Estimated delivery time

Could Have

- Promo codes and discounts

- Multiple payment methods
- Favorite restaurants and items
- In-app customer support chat

Won't Have (for now)

- AI-based recommendations
 - Dynamic pricing
 - Advanced analytics and reporting
 - Multi-country tax and compliance handling
-

Method

High-Level Architecture (Phase 1 – MVP)

We will begin with a **Modular Monolith Architecture**. This approach keeps deployment and operations simple while enforcing clear internal boundaries so the system can later evolve into microservices if needed.

Core Architectural Components

1. Client Applications

2. Customer Mobile App (Android / iOS)
3. Restaurant Web Dashboard
4. Delivery Partner Mobile App
5. Admin Web Panel

6. Backend Application (Single Codebase)

7. Authentication & Authorization Module
8. User Management Module
9. Restaurant & Menu Module
10. Order Management Module
11. Payment Module (Gateway integration)
12. Delivery & Tracking Module
13. Notification Module

14. Data Layer

15. Relational Database (primary system of record)
16. In-memory cache (for frequently accessed data – optional in MVP)

17. External Services

18. Payment Gateway
 19. Push Notification Service (SMS / Email / App)
-

Logical Architecture Diagram (Conceptual)

```
@startuml
actor Customer
actor RestaurantOwner
actor DeliveryPartner
actor Admin

Customer --> MobileApp
RestaurantOwner --> WebDashboard
DeliveryPartner --> DeliveryApp
Admin --> AdminPanel

MobileApp --> Backend
WebDashboard --> Backend
DeliveryApp --> Backend
AdminPanel --> Backend

Backend --> Database
Backend --> PaymentGateway
Backend --> NotificationService

@enduml
```

Why Modular Monolith?

- Easier to understand for beginners
- Faster development and deployment
- Avoids premature microservice complexity
- Clear module boundaries prepare for future service extraction

Core User Flow: Customer Places an Order

This section describes the **end-to-end flow** when a customer places a food order. This flow drives most backend design decisions (data models, APIs, transactions).

Step-by-Step Flow

1. Customer opens the mobile app
2. App fetches nearby restaurants based on location
3. Customer selects a restaurant
4. App fetches menu items for the restaurant
5. Customer adds items to cart
6. Customer proceeds to checkout
7. Customer selects delivery address
8. Customer makes payment
9. Order is created in the system

10. Restaurant receives the order
 11. Delivery partner is assigned
 12. Order status updates until delivery
-

Order Placement Sequence Diagram

```
@startuml
actor Customer
participant MobileApp
participant Backend
participant PaymentGateway
participant Restaurant
participant DeliveryPartner

Customer -> MobileApp : Browse & Add to Cart
MobileApp -> Backend : Fetch Restaurants & Menu

Customer -> MobileApp : Checkout
MobileApp -> Backend : Create Order (Pending Payment)
Backend -> PaymentGateway : Initiate Payment
PaymentGateway --> Backend : Payment Success

Backend -> Backend : Confirm Order
Backend -> Restaurant : Notify New Order
Backend -> DeliveryPartner : Assign Delivery

DeliveryPartner -> Backend : Update Status
Backend -> MobileApp : Order Status Updates
@enduml
```

Key Design Decisions from This Flow

- Order creation must be **transactional**
 - Payment confirmation is mandatory before order confirmation
 - Order status must be persisted and auditable
 - Restaurant and delivery partner interactions are asynchronous from customer perspective
-

Data Models & Database Schema

The database schema is designed **directly from the order placement flow**. We start with minimal, normalized tables to support MVP requirements while remaining extensible.

Database Choice

- **Relational Database:** PostgreSQL (recommended)
- ACID compliance is critical for orders and payments

Core Tables (MVP)

1. users

Stores all system users (customers, restaurant owners, delivery partners, admins).

Column	Type	Notes
id	UUID	Primary Key
name	VARCHAR	
email	VARCHAR	Unique
phone	VARCHAR	Unique
role	ENUM	CUSTOMER, RESTAURANT, DELIVERY, ADMIN
password_hash	VARCHAR	Encrypted
created_at	TIMESTAMP	

2. restaurants

Column	Type	Notes
id	UUID	Primary Key
name	VARCHAR	
owner_id	UUID	FK → users.id
address	TEXT	
is_active	BOOLEAN	
created_at	TIMESTAMP	

3. menu_items

Column	Type	Notes
id	UUID	Primary Key
restaurant_id	UUID	FK → restaurants.id
name	VARCHAR	
price	DECIMAL	
is_available	BOOLEAN	

4. orders

Column	Type	Notes
id	UUID	Primary Key
customer_id	UUID	FK → users.id
restaurant_id	UUID	FK → restaurants.id
status	ENUM	PLACED, PREPARING, PICKED_UP, DELIVERED
total_amount	DECIMAL	
created_at	TIMESTAMP	

5. order_items

Column	Type	Notes
id	UUID	Primary Key
order_id	UUID	FK → orders.id
menu_item_id	UUID	FK → menu_items.id
quantity	INT	
price	DECIMAL	Price at time of order

6. payments

Column	Type	Notes
id	UUID	Primary Key
order_id	UUID	FK → orders.id
status	ENUM	INITIATED, SUCCESS, FAILED
provider	VARCHAR	Stripe/Razorpay/etc
transaction_ref	VARCHAR	Gateway reference
created_at	TIMESTAMP	

Entity Relationship Diagram

```
@startuml
entity users
entity restaurants
```

```
entity menu_items
entity orders
entity order_items
entity payments

users ||--o{ restaurants : owns
restaurants ||--o{ menu_items
users ||--o{ orders : places
orders ||--o{ order_items
menu_items ||--o{ order_items
orders ||--|| payments
@enduml
```

Key Design Notes

- Single users table simplifies authentication
- Prices stored in order_items to prevent historical price mismatch
- Payments separated for auditability
- ENUMs keep state management explicit

Backend Modules & API Design

This section defines **clear backend module boundaries** and **REST API contracts** that directly operate on the database schema and support the core order flow.

Backend Module Structure (Logical)

- Auth Module
- User Module
- Restaurant Module
- Menu Module
- Order Module
- Payment Module
- Delivery Module
- Notification Module

Each module owns its data access and business rules.

API Design Principles

- RESTful endpoints
 - JSON request/response
 - Stateless authentication (JWT)
 - Idempotent order & payment APIs
-

Core APIs (MVP)

Authentication

- POST /auth/register
 - POST /auth/login
-

Restaurants & Menus

- GET /restaurants?lat=&lng=
 - GET /restaurants/{id}
 - GET /restaurants/{id}/menu
-

Orders

- POST /orders
- GET /orders/{id}
- GET /users/{id}/orders
- PATCH /orders/{id}/status

Create Order (POST /orders)

```
{
  "restaurant_id": "uuid",
  "items": [
    { "menu_item_id": "uuid", "quantity": 2 }
  ],
  "delivery_address": "string"
}
```

Payments

- POST /payments/initiate
 - POST /payments/confirm
-

Delivery

- POST /deliveries/assign
 - PATCH /deliveries/{orderId}/status
-

Order Creation Algorithm (Simplified)

```
@startuml
start
:Validate cart;
:Calculate total;
:Create order (PENDING_PAYMENT);
:Initiate payment;
if (Payment success?) then (yes)
    :Confirm order;
    :Notify restaurant;
else (no)
    :Cancel order;
endif
stop
@enduml
```

Key Design Notes

- Order creation is idempotent
- Payment confirmation is webhook-driven
- Status transitions are validated centrally

Scalability, Security & Deployment Architecture

This section focuses on making the system **production-ready**, ensuring it is secure, scalable, and deployable with confidence.

Deployment Architecture (MVP)

- Cloud Provider: AWS / GCP / Azure (cloud-agnostic design)
- Containerization: Docker
- Backend deployed as a single service (modular monolith)
- Managed relational database (PostgreSQL)
- Object storage for images (menus, restaurant logos)

```
@startuml
node "Client Apps" {
    component MobileApp
    component WebDashboard
}

node "Cloud" {
    node "Load Balancer" {
        component LB
    }
}
```

```
}  
node "App Service" {  
    component Backend  
}  
node "Database" {  
    component PostgreSQL  
}  
}  
  
MobileApp --> LB  
WebDashboard --> LB  
LB --> Backend  
Backend --> PostgreSQL  
@enduml
```

Scalability Strategy

What scales first? - Read-heavy endpoints (restaurants, menus) - Order creation during peak hours

MVP Solutions: - Horizontal scaling of backend containers - Database indexing (restaurant_id, user_id, status) - Optional in-memory caching for menus

Security Fundamentals

- HTTPS everywhere
 - JWT-based authentication
 - Role-based authorization (RBAC)
 - Password hashing (bcrypt/argon2)
 - Payment handled only via gateway (no card storage)
 - Webhook signature validation
-

Failure Handling

- Payment failures → order auto-cancel
 - Retry mechanisms for notifications
 - Graceful degradation (read-only mode if DB is slow)
-

Implementation

The implementation is divided into **clear, sequential phases** to minimize risk and enable parallel work where possible.

Phase 1: Foundation (Week 1-2)

- Finalize requirements & architecture

- Setup repositories and CI/CD pipeline
- Configure cloud infrastructure
- Setup database and migrations

Phase 2: Backend Core (Week 3–4)

- Authentication & user management
- Restaurant and menu modules
- Order & payment modules
- Integration with payment gateway

Phase 3: Client Applications (Week 5–6)

- Customer mobile app (core flows)
- Restaurant web dashboard
- Delivery partner app (basic)

Phase 4: Integration & Hardening (Week 7–8)

- End-to-end testing
- Load testing (basic)
- Security review
- Bug fixes

Milestones

Milestone	Outcome
Architecture Approved	Team aligned
Backend APIs Ready	Frontend unblocked
Order Flow Live	Core value delivered
MVP Feature Complete	Ready for pilot
Production Launch	Real users onboarded

Gathering Results

Post-production evaluation focuses on both **technical** and **business** metrics.

Technical Metrics

- API latency (p95)
- Order failure rate
- Payment success rate
- System uptime

Business Metrics

- Orders per day
- Conversion rate
- Average delivery time
- Customer retention

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)